10 Policy as code: Effective authorization policies

This chapter covers

- How to design policies that are clear, least-privilege, and resilient to change
- Why treating policies as code makes them easier to review, version, and deploy
- How to manage flow from policy repositories to policy stores without confusion
- Methods for testing policies with real-life scenarios to prevent bugs from entering production
- How static analysis provides a safety net by catching conflicts and gaps you might overlook

At ACME Corp, the identity team has rolled out dynamic authorization across multiple systems. Employees authenticate via single sign-on, and the Customer Collaboration platform enforces ReBAC so that only project participants can access shared documents. The engineering team uses ABAC to manage production deployments. Policies are now a core part of how the company controls access. However, as the team increases its use of policies, a new challenge arises: how to make these policies effective and sustainable over time.

Writing a policy that works once is easy; creating a policy that remains relevant as employees change roles, projects reorganize, or compliance rules evolve is more challenging. The identity team understands that good policies are not just statements of permission and denial; they are design artifacts that must be clear, consistent, and aligned with organizational goals.

At the same time, policies can’t stay abstract. They need to be enforced by running systems, and ACME’s developers need a way to manage them